

LAPORAN PRAKTIKUM

Praktikum-W4

Fundamental Programming Structures in Java

Diajukan untuk memenuhi salah satu tugas praktikum Mata kuliah Pemrograman berbasis objek



Disusun Oleh:
M. Fachri Akbar Irfan (251511045)
Jurusan Teknik Komputer dan Informatika

02-10-2026
Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung 2026

1. Exercise 1 – circle dan cylinder

Pada Exercise 1, pembahasan berfokus pada hubungan inheritance antara class Circle dan Cylinder. Dalam kasus ini, Circle berperan sebagai superclass, sedangkan Cylinder merupakan subclass yang mewarisi beberapa atribut dan method dari Circle.

Melalui latihan ini, dapat dipahami bahwa subclass tidak perlu menuliskan kembali seluruh atribut dan method yang sudah dimiliki superclass. Subclass cukup menggunakan extends untuk memperoleh behavior yang memang dapat diwarisi. Selain itu, penggunaan super() dan super.method() juga diperkenalkan untuk mengakses constructor maupun method milik superclass.

1.1 Task 1.1 - Modify class Circle

Pada Task 1.1, class Circle dikembangkan dengan menambahkan constructor yang dapat menerima nilai radius dan color sekaligus. Selain itu, dibuat juga method getter dan setter untuk atribut color.

Atribut pada Circle tetap dibuat menggunakan access modifier private. Hal ini dilakukan agar data di dalam object tidak dapat diubah secara langsung dari luar class. Jika ingin membaca atau mengubah nilai tersebut, penggunaannya dilakukan melalui method yang telah disediakan.

Dengan demikian, task ini tidak hanya membahas penambahan constructor, tetapi juga memperlihatkan penerapan konsep encapsulation.

Contoh constructor yang digunakan:

```
public Circle(double radius, String color) {  
    this.radius = radius;  
    this.color = color;  
}
```

Method untuk membaca dan mengubah warna dapat dibuat seperti berikut:

```
public String getColor() {  
    return color;  
}  
  
public void setColor(String color) {  
    this.color = color;  
}
```

1.2 Task 1.2 - Overriding getArea()

Pada task ini, method getArea() yang sebelumnya berasal dari Circle dioverride oleh class Cylinder.

Pada Circle, method getArea() digunakan untuk menghitung luas lingkaran. Namun, pada Cylinder, method dengan nama yang sama diubah fungsinya menjadi penghitung luas permukaan silinder.

Hal tersebut menunjukkan bahwa subclass dapat memiliki implementasi berbeda terhadap method yang diwarisi dari superclass.

Permasalahan muncul pada method `getVolume()`. Sebelum terjadi overriding, volume dihitung menggunakan:

`getArea() * height`

Pada kondisi awal, `getArea()` masih mengembalikan luas alas lingkaran sehingga perhitungan volume benar.

Setelah `getArea()` dioverride di `Cylinder`, pemanggilan `getArea()` akan menggunakan versi milik `Cylinder`, yaitu luas permukaan silinder. Akibatnya, rumus volume menjadi tidak sesuai.

Untuk mengatasinya, method `getVolume()` harus secara khusus memanggil `getArea()` milik superclass `Circle`:

```
public double getVolume() {  
    return super.getArea() * height;  
}
```

Penggunaan `super.getArea()` berarti program secara eksplisit meminta implementasi `getArea()` yang dimiliki oleh `Circle`, bukan versi yang telah dioverride oleh `Cylinder`.

```
c1  
radius = 1.0  
height = 1.0  
base area = 12.566370614359172  
volume = 3.141592653589793  
  
c2  
radius = 1.0  
height = 10.0  
base area = 69.11503837897544  
volume = 31.41592653589793  
  
c3  
radius = 2.0  
height = 10.0  
base area = 150.79644737231007  
volume = 125.66370614359172
```

1.3 Task 1.3 - `toString()` dan `@Override`

Pada Task 1.3, class `Cylinder` melakukan overriding terhadap method `toString()`.

Method `toString()` digunakan untuk menghasilkan representasi suatu object dalam bentuk teks. Karena `Cylinder` merupakan turunan dari `Circle`, informasi milik `Circle` tetap dapat digunakan melalui:

`super.toString()`

Contohnya:

`@Override`

```
public String toString() {  
    return "Cylinder: subclass of " + super.toString()  
        + ", height=" + height;  
}
```

```
}
```

Annotation `@Override` digunakan untuk memberi tahu compiler bahwa method yang ditulis memang dimaksudkan untuk menggantikan method yang telah tersedia pada superclass.

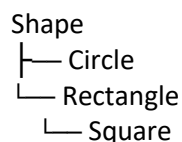
`@Override` tidak menyebabkan inheritance terjadi. Inheritance sudah terbentuk melalui `extends`. Fungsi `@Override` adalah membantu compiler memeriksa apakah nama, parameter, dan bentuk method sudah benar-benar sesuai dengan method yang ingin dioverride.

Dengan demikian, jika terdapat kesalahan penulisan nama method, compiler dapat langsung mendeteksinya.

2. Exercise 2 - Shape, Circle, Rectangle, dan Square

Exercise 2 membahas inheritance dengan struktur class yang lebih besar. Dalam latihan ini, beberapa bentuk geometri memiliki karakteristik umum yang ditempatkan pada sebuah superclass bernama Shape.

Struktur inheritance yang digunakan adalah:



Shape menjadi class yang paling umum. Circle dan Rectangle menjadi bentuk yang lebih spesifik, sedangkan Square merupakan bentuk yang lebih khusus dari Rectangle.

Hasil akhir dari program exercise dua sebagai berikut

```
Problems 4 Output Debug Console Terminal Ports
setWidth(8.0)      : Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
setLength(3.0)     : Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
area akhir         : 9.0
perimeter akhir    : 12.0
```

2.1 Class Shape sebagai Superclass

Class Shape menyimpan atribut yang dianggap umum untuk beberapa bentuk, yaitu:

color
filled

Atribut color digunakan untuk menyimpan warna object, sedangkan filled menunjukkan apakah suatu bentuk memiliki isi atau tidak.

Karena kedua informasi tersebut dapat dimiliki oleh Circle maupun Rectangle, atribut tersebut ditempatkan pada superclass Shape.

Dengan cara ini, kode yang sama tidak perlu ditulis berulang kali pada setiap subclass.

2.2 Circle sebagai Subclass Shape

Class Circle merupakan subclass dari Shape.

Selain mewarisi color dan filled, Circle memiliki atribut khusus berupa:

radius

Class ini juga mempunyai method yang khusus digunakan untuk lingkaran, seperti:

```
getArea()  
getPerimeter()
```

Dengan demikian, Circle memanfaatkan karakteristik umum dari Shape, tetapi tetap memiliki state dan behavior yang sesuai dengan bentuk lingkaran.

2.3 Rectangle sebagai Subclass Shape

Sama seperti Circle, Rectangle juga merupakan subclass dari Shape.

Perbedaannya, Rectangle memiliki atribut khusus:

width
length

Dari kedua atribut tersebut, luas dan keliling dapat dihitung menggunakan:

```
getArea()  
getPerimeter()
```

Walaupun Circle dan Rectangle sama-sama berasal dari Shape, keduanya tetap memiliki implementasi yang berbeda karena masing-masing mempunyai karakteristik bentuk yang berbeda.

2.4 Square sebagai Subclass Rectangle

Square atau persegi dibuat sebagai subclass dari Rectangle.

Hal ini dapat dilakukan karena sebuah persegi pada dasarnya dapat dianggap sebagai persegi panjang yang memiliki nilai:

$width = length$

Oleh karena itu, Square tidak perlu membuat atribut baru. Nilai sisi dapat memanfaatkan width dan length yang sudah tersedia pada Rectangle.

Namun, terdapat satu masalah.

Jika Square menggunakan setter milik Rectangle secara langsung, misalnya:

```
setWidth(8.0);
```

maka hanya width yang berubah, sedangkan length masih memiliki nilai sebelumnya. Akibatnya, object tersebut tidak lagi memenuhi aturan sebuah persegi.

Untuk menjaga agar width dan length selalu sama, Square perlu meng-override method `setWidth()` dan `setLength()`.

Contohnya:

```
@Override
public void setWidth(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override
public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}
```

Dengan cara tersebut, ketika salah satu nilai sisi diubah, kedua nilai tetap dibuat sama.

Sementara itu, method `getArea()` dan `getPerimeter()` tidak perlu dioverride. Selama width dan length selalu sama, rumus yang dimiliki Rectangle tetap dapat digunakan untuk menghitung luas dan keliling persegi dengan benar.

3. Exercise 3 – Employee, Manager, dan Sortable

Exercise 3 membahas inheritance melalui kasus Employee dan Manager, kemudian dikembangkan dengan penggunaan abstract class bernama Sortable.

Bagian ini berbeda dengan Exercise 1 dan Exercise 2 karena mulai memperkenalkan konsep abstract class, abstract method, pengurutan object, serta inheritance bertingkat.

Implementasi `shell_sort()` yang digunakan merupakan pelengkapan kode karena pada soal aslinya bagian tersebut hanya diberikan dalam bentuk komentar `//Shell sort body`.

Pengujian `employetest.java`

```
Employee setelah kenaikan 5% dan diurutkan berdasarkan salary:
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
```

Pengujian dari ManagerTest.java

```
s' '-cp' 'C:\Users\DELL\AppData\Roaming\Code\User\workspaceStorage\89f4ea
Employee/Manager setelah kenaikan dan diurutkan berdasarkan salary:
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
```

3.1 Analisis Class Employee dan Manager

Class Employee digunakan untuk merepresentasikan seorang pegawai.

Informasi yang disimpan antara lain:

name
salary
hireday
hiremonth
hireyear

Method raiseSalary() digunakan untuk meningkatkan salary berdasarkan persentase tertentu.

Contohnya, apabila gaji seorang Employee adalah Rp2.000.000 dan diberikan kenaikan 5%, maka nilai salary akan menjadi Rp2.100.000.

Selain itu, method hireYear() digunakan untuk mendapatkan tahun mulai bekerja.

Class Manager kemudian dibuat sebagai subclass dari Employee:

```
class Manager extends Employee
```

Hubungan tersebut berarti bahwa seorang Manager juga merupakan Employee.

Constructor Manager menggunakan:

```
super(...)
```

untuk memanggil constructor milik Employee. Dengan demikian, proses inisialisasi atribut yang berasal dari Employee tetap dilakukan oleh class Employee.

Manager kemudian meng-override method raiseSalary().

Pada Employee, kenaikan hanya berdasarkan persentase yang diberikan. Sementara itu, Manager mendapatkan tambahan bonus berdasarkan lama bekerja.

Contohnya:

```

@Override
public void raiseSalary(double byPercent) {
    GregorianCalendar todaysDate = new GregorianCalendar();

    int currentYear =
        todaysDate.get(Calendar.YEAR);

    double bonus =
        0.5 * (currentYear - hireYear());

    super.raiseSalary(byPercent + bonus);
}

```

Pada bagian tersebut, Manager menghitung bonus terlebih dahulu. Setelah itu:

```
super.raiseSalary(...)
```

digunakan untuk menjalankan mekanisme perubahan salary yang sudah dimiliki Employee.

Dengan cara ini, Manager tidak perlu menuliskan ulang seluruh proses kenaikan gaji.

3.2 Sortable sebagai Abstract Class

Sortable dibuat sebagai abstract class:

```
abstract class Sortable
```

Abstract class merupakan class yang digunakan sebagai dasar bagi class lain dan tidak harus dibuat object secara langsung.

Pada Sortable terdapat abstract method:

```
public abstract int compare(Sortable b);
```

Method tersebut belum mempunyai implementasi di dalam Sortable.

Artinya, Sortable hanya memberikan aturan bahwa class yang menjadi subclass-nya harus mengetahui bagaimana cara membandingkan object dengan object lain.

Selain itu, Sortable mempunyai method:

```
shell_sort()
```

yang digunakan untuk melakukan proses pengurutan.

Secara sederhana, pembagian tugasnya adalah:

Sortable

→ menyediakan mekanisme pengurutan

Employee

→ menentukan bagaimana dua Employee dibandingkan

Pada kasus ini, Employee dibandingkan berdasarkan nilai salary.

3.3 Employee sebagai Subclass Sortable

Class Employee kemudian diubah menjadi:

```
class Employee extends Sortable
```

Karena Employee merupakan class konkret dan Sortable memiliki abstract method compare(), Employee harus menyediakan implementasi method tersebut.

Contohnya:

```
@Override
public int compare(Sortable b) {
    Employee eb = (Employee) b;

    if (salary < eb.salary)
        return -1;

    if (salary > eb.salary)
        return 1;

    return 0;
}
```

Nilai yang dikembalikan memiliki arti:

-1 → salary object sekarang lebih kecil
0 → kedua salary sama
1 → salary object sekarang lebih besar

Dengan aturan tersebut, shell_sort() dapat menentukan posisi object Employee berdasarkan salary.

Hasil pengurutannya adalah dari nilai salary yang lebih kecil menuju nilai yang lebih besar atau **ascending**.

3.4 Pengujian Case 1

Pada pengujian Case 1 dibuat sebuah array Employee:

```
Employee[] staff = new Employee[3];
```

Kemudian beberapa object Employee dimasukkan ke dalam array tersebut.

Setelah itu dijalankan:

```
Sortable.shell_sort(staff);
```

Pemanggilan tersebut dapat dilakukan karena Employee merupakan subclass dari Sortable.

Hubungannya dapat digambarkan sebagai berikut:

```
Sortable
  ↑
Employee
```

Karena Employee termasuk Sortable, kumpulan object Employee dapat diberikan kepada method pengurutan yang menerima Sortable[].

Saat proses pengurutan berlangsung, shell_sort() menggunakan method compare() milik Employee untuk menentukan urutan object berdasarkan salary.

3.5 Case 2 – Apakah extends Employee extends Sortable Dapat Digunakan?

Pada Case 2 terdapat contoh:

```
class Manager extends Employee extends Sortable
```

Bentuk tersebut tidak dapat digunakan pada Java karena syntax tersebut tidak valid.

Selain itu, penulisan tersebut sebenarnya juga tidak diperlukan.

Jika Employee sudah dibuat sebagai:

```
class Employee extends Sortable
```

dan Manager dibuat sebagai:

```
class Manager extends Employee
```

maka hubungan class akan menjadi:

```
Sortable
  ↓
Employee
  ↓
Manager
```

Dengan hubungan tersebut, Manager secara tidak langsung juga merupakan Sortable.

Hal ini terjadi karena Manager mewarisi Employee, sedangkan Employee sebelumnya telah mewarisi Sortable.

Jadi Manager tidak perlu kembali menuliskan:

`extends Sortable`

Hubungan seperti ini disebut **multilevel inheritance**, yaitu inheritance yang terjadi secara bertingkat.

Contohnya:

A
↓
B
↓
C

Berbeda dengan **multiple inheritance**, yaitu ketika satu class mencoba mewarisi lebih dari satu superclass secara langsung.

Java tidak mendukung multiple inheritance antar class.

Apabila sebuah class membutuhkan beberapa kemampuan tambahan, Java biasanya menggunakan interface. Namun, untuk kasus ini, hierarchy:

`Sortable → Employee → Manager`

sudah mencukupi.

3.6 Manager.java agar Dapat Dikompilasi

Pada Manager.java, digunakan class:

Calendar
GregorianCalendar

Oleh karena itu, bagian import perlu ditambahkan:

```
import java.util.Calendar;  
import java.util.GregorianCalendar;
```

Tanpa import tersebut, compiler tidak akan mengenali Calendar dan GregorianCalendar.

Struktur utama Manager menjadi:

```
class Manager extends Employee
```

Manager mewarisi behavior Employee, termasuk implementasi `compare()`.

Dengan demikian, Manager juga dapat diperlakukan sebagai object yang dapat diurutkan tanpa perlu mengimplementasikan kembali method `compare()`.

4. Ringkasan Observasi dari Hasil Run

Pengujian	Observasi yang diharapkan	Bukti yang dilampirkan
Circle/Cylinder constructor chain	Constructor superclass berjalan lebih dahulu sebelum bagian constructor subclass.	Screenshot output constructor sementara.
Akses field private	Subclass tidak dapat mengakses radius private secara langsung; akses dilakukan melalui method public/protected yang sesuai.	Screenshot error compiler.
Override getArea()	getArea() pada Cylinder menjalankan versi Cylinder; super.getArea() memanggil versi Circle.	Output sebelum/sesudah override.
Square setter	Setter Rectangle dapat merusak syarat width == length; override di Square menjaga keduanya tetap sama.	Output sebelum/sesudah override setter.
Sortable shell_sort	Employee dapat diurutkan karena mengimplementasikan compare() dari abstract class Sortable.	Output EmployeeTest.
Manager inheritance	Manager cukup extends Employee karena Employee sudah extends Sortable.	Source final Manager dan hasil compile/run.

5. Kesimpulan

Berdasarkan praktikum yang dilakukan, inheritance memungkinkan sebuah subclass menggunakan kembali state dan behavior yang telah tersedia pada superclass tanpa harus menuliskan kode yang sama berulang kali.

Keyword `extends` digunakan untuk membentuk hubungan inheritance. Sementara itu, `super(...)` digunakan untuk memanggil constructor superclass ketika object subclass dibuat.

Jika subclass meng-override sebuah method tetapi masih membutuhkan implementasi dari superclass, maka dapat digunakan:

```
super.method()
```

Annotation `@Override` membantu memastikan bahwa method yang dibuat benar-benar menggantikan method yang tersedia pada superclass.

Pada kasus Shape, inheritance digunakan untuk menempatkan karakteristik yang umum pada superclass dan karakteristik khusus pada masing-masing subclass. Kasus Square juga menunjukkan bahwa tidak semua behavior yang diwarisi selalu sesuai secara langsung dengan aturan subclass sehingga overriding terkadang diperlukan.

Pada Exercise 3, abstract class Sortable memberikan aturan bahwa object yang dapat diurutkan harus memiliki method `compare()`. Employee mengimplementasikan aturan tersebut dengan membandingkan salary.

Manager tidak perlu melakukan inheritance langsung lagi terhadap Sortable karena hubungan yang terbentuk sudah berupa:

```
Sortable
  ↓
Employee
  ↓
Manager
```

Hubungan tersebut merupakan multilevel inheritance. Dengan struktur tersebut, Manager tetap menjadi Employee sekaligus memperoleh kemampuan yang berasal dari Sortable tanpa menggunakan multiple inheritance antar class.